Verified repair is not finished until visitors receive it.

Sugamya: A Self-Verifying AI Agent That Fixes Web Accessibility

An owner-controlled accessibility repair control plane.

No target source patch. No visitor extension.

4 → 0	0/3 → 3/3	11 / 11
Seeded axe instances	**Automated service-task trials**	**Public visitor gates**
One owned public fixture; not conformance	Same accessible-name + keyboard method	Fresh browser, anonymous HTTPS

The bottleneck is safe delivery, not another list of issues.

A scanner or a patch on a copy does not change what an ordinary citizen receives.

Purchaser	Service-owning agency or managed-service provider
Operator	Small web operations / infrastructure team
Approver	Accessibility or content owner; decides meaning
Visitor	Uses the service normally; installs nothing

Required authority: control the origin installation and visitor routing—not access to application source.

We are not claiming the first accessibility agent.

The differentiated proof is the bounded change-control loop, not AI, PRs, alt text, or runtime repair alone.

GitHub agent / Scanner	Objective code remediation, issues, Copilot PR workflow
GitHub alt-text plugin	Deterministic quality rules + opt-in image/context model review
AudioEye	Site-installed runtime fixes plus expert/custom remediation
axe / enterprise platforms	Strong automated evidence, guided tests and governance

Our claim: inspectable owner-controlled manifests + human semantics + independent visitor gates + replayable failures.

GitHub agent + scanner: current primary sources

AudioEye: current runtime description

Full cited matrix in repository

Owner-installed proxy. Approved manifest. Real visitors.

The repair engine consumes HTTP evidence; the original application remains unchanged.

Separate origin

Owned public HTML

Existing CSP + form endpoint

→

Sugamya gateway

Exact page fingerprint

Allowlisted attributes/class

→

Ordinary browser

No extension or operator state

Version header + completed task

Observe → prioritize → semantic review → canary → activate → fresh-browser verify → keep / rollback

Temporary Quick Tunnel proves public HTTPS reach; not production hosting.

Contrast uses an existing owner-approved utility class; dynamic/private pages are outside the released repair scope.

55 criteria inventoried does not mean 55 automated checks.

WCAG 2.2 AA · applicability and test methods are disclosed, not converted into a compliance score.

1	3	3	5	43
Selected automated	**Human-assisted**	**Manual remainder**	**Inapplicable here**	**Unresolved / untested**
Rendered contrast evidence	Alt, labels, name/role/value	Full keyboard/focus assessment	No media in this static state	No criterion-wide assessment

Priority favors the blocked application task—not easy-to-count rules.

Severity × confidence × evaluated-task reach ÷ estimated review effort. Reach and effort are disclosed assumptions.

No universal “20 manual criteria” partition was verified. No whole-criterion or site conformance claim.

W3C WCAG 2.2 and WAI evaluation-tool limits

The model drafts text. A human decides meaning.

One real Groq request used only redacted nearby text; no image or screenshot went to a model.

Actual AI draft

“Diagram showing route from entrance to service desk.”

Uncertainty: “Based on provided context; no visual details confirmed.”

Human edited and explicitly approved

“From Entrance, follow the path right, turn upward, then right to Service desk.”

All 3 semantic operations bind approval to the exact manifest; edits clear prior approvals.

441 tokens · 380 ms observed · billed cost unverified. Provider retention exceptions may apply; ZDR not verified.

Signed-in routes denied; no screenshot, image, full HTML, form value or credential in the model payload.

The improved task reaches a public, clean browser.

Recorded evidence: actual HTTPS proxy response, accessible-name snapshot, keyboard submission and unchanged origin.

4 → 0

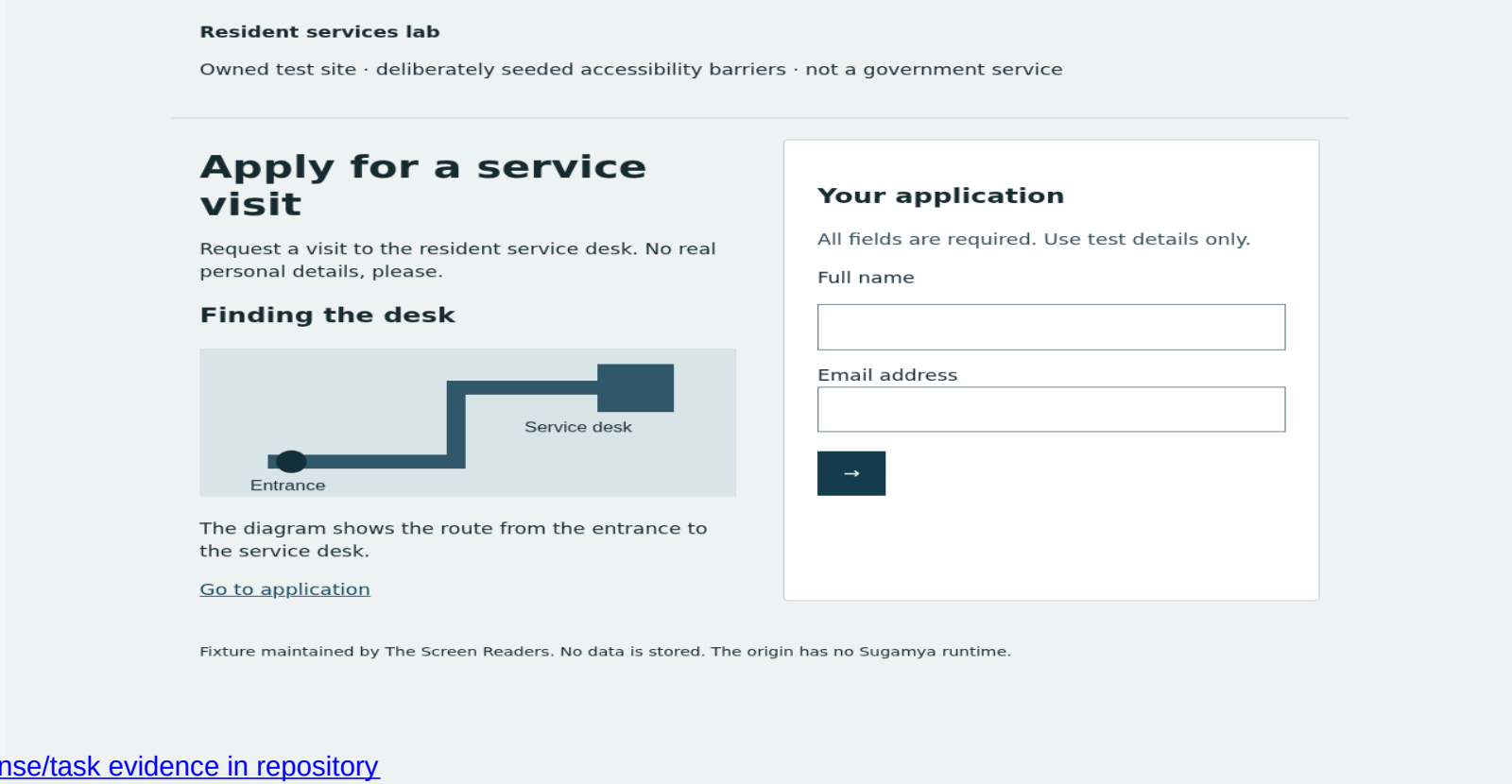
Seeded axe instances

1 page, 4 instances; no conformance inference

0/3 → 3/3

Same automated task

Fresh context per trial; not a screen-reader user study



Temporary public proof URL (may expire) · full response/task evidence in repository

Passing a scan is not permission to promote.

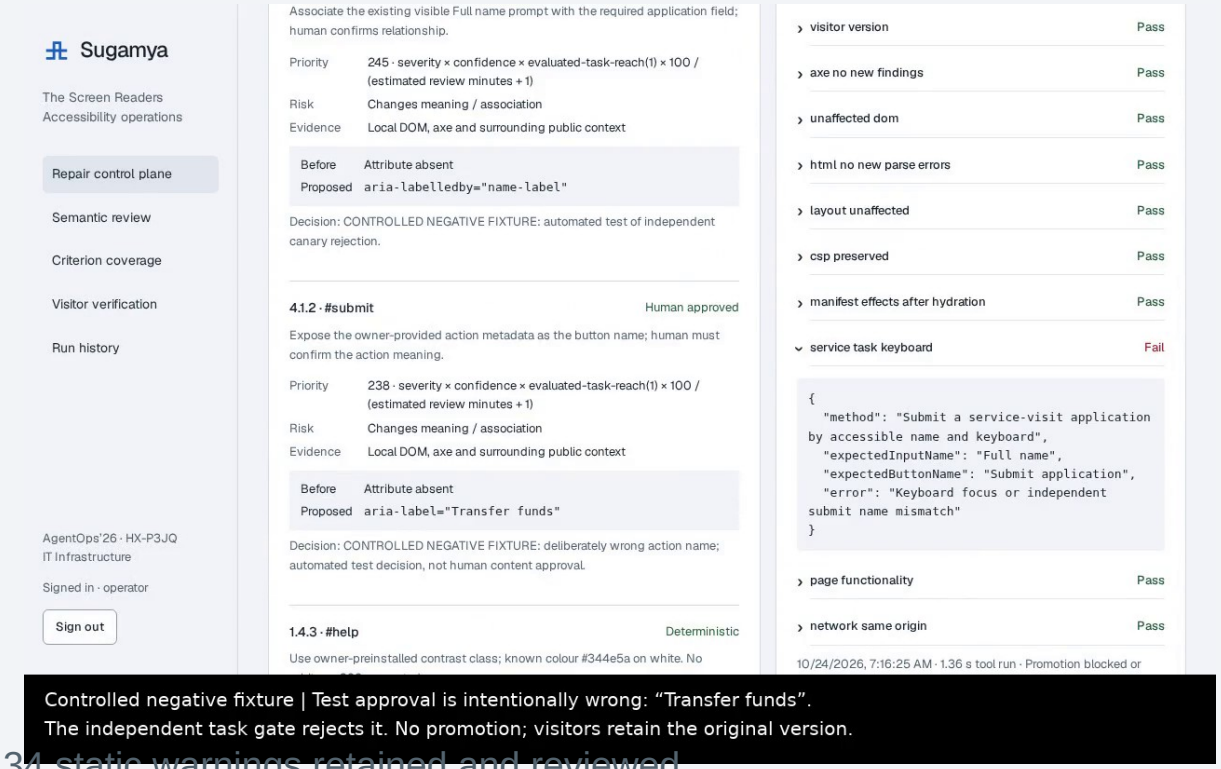
A deliberately wrong approved test label passed syntax/axe checks—but failed the independent task gate.

Controlled negative canary

“Transfer funds” instead of

“Submit application” → withheld.

Separate stale-origin test: fingerprint mismatch → original response → failed independent check → actual active-pointer rollback.



Controlled negative fixture | Test approval is intentionally wrong: “Transfer funds”.
The independent task gate rejects it. No promotion; visitors retain the original version.

30 unit tests · 21 E2E checks · 0 known dependency advisories · 34 static warnings retained and reviewed.

Measure the workflow. Label the forecast.

Observed execution evidence and assumed business savings are kept separate.

4.78 ms

Median added local latency

30 paired warm requests; proxy + parser + local network

₹3,000 / mo

Illustrative effort value

10 sites × (45−15 min) × ₹600/hour; assumptions

₹300 / ₹1,500

Sensitivity cases

1 site / doubled review burden; hosting and audits excluded

Human review/setup time was not timed. No measured saving or customer ROI is claimed.

Second installed target + separate origin/gateway/audit store passed the same 21 checks.

Main model call: 441 tokens; post-hoc public-list-price estimate ≈ US$0.0000668, not an invoice.

A narrow accelerator judges can inspect and rerun.

One repeatable contract: owner authority + approved change + visitor proof + operational reversal.

Reuse	Configuration, strict contracts and a second independent installation
Governance	Explicit semantic consent/approval, minimal evidence, roles and trace
Business value	Task outcomes and measured latency; forecast-only effort model
MVP readiness	Actual public route, tested controls, MP4, reproducible repository
Innovation	The inspectable delivery/verification combination—not AI or overlays alone

https://github.com/shivanshjain456/Sugamya-TheScreenReaders

Limits: fixture only · static public HTML · existing contrast class · no tenant isolation/SSO · temporary tunnel · no conformance claim

What we will not claim.

Five-minute Q&A preparation; not part of the five-minute slide narrative.

“Is this source repair?”	No. It transforms eligible HTTP responses. Origin source stays unchanged; removing the route removes repairs.
“Why not GitHub?”	Use GitHub when the repo-owning team can merge and deploy. Our route addresses a different permission bottleneck; no superiority claim.
“Can it understand every image?”	No image is sent in our model path. Human review supplies visual meaning; insufficient context stays unresolved.
“What about all 55 criteria?”	A method-disclosing inventory is not coverage. 43 untested and 3 fuller-manual criteria remain open.
“Production-ready for any site?”	No. Owned static fixture release; stronger isolation, client routing, new task policies and manual audits remain prerequisites.

Evidence, commands and fallback.

Use the repository artifacts to replay observations; use actual live controls only when the environment is running.

evidence/public-deployment-proof.json — ordinary HTTPS browser, version and task

evidence/human-approved-deployment.json — explicit project-owner semantic decision

evidence/reuse/e2e-results.json — 21-case independent reuse/failure suite

evidence/model-live-result.json — actual consented request/output and held deployment

evidence/measurements.json — denominators, 30 latency pairs and 3+3 task trials

npm test · npm run test:e2e · npm run test:ui · npm run replay

artifacts/Sugamya-demo.mp4 — actual 79.6-second captioned recording, not live

Setup / architecture / security / coverage / competitor primary sources